C PROGRAMMING

Lesson 7 — If Statements

Based on: Bro Code C Tutorial Series

1. What is an If Statement?

An if statement lets your program make decisions. It runs a block of code only if a condition is true. If the condition is false, that block is skipped entirely.

```
int age = 20;

if (age >= 18) {
    printf("You are an adult\n");   // runs only if age >= 18
}
```

- **The condition** goes inside parentheses ( )
- **The code to run** goes inside curly braces { }
- **If the condition is false,** everything inside { } is skipped

2. if / else

The else block runs when the if condition is false. Think of it as the fallback — "if this, otherwise that".

```
int age = 15;

if (age >= 18) {
    printf("You are an adult\n");
} else {
    printf("You are a child\n");   // runs because age < 18
}
```

3. if / else if / else

Use else if to check multiple conditions one after another. C evaluates them top to bottom and stops at the first true condition.

```
int age = 70;

if (age < 0) {
    printf("You haven't been born yet\n");
} else if (age == 0) {
    printf("You are a newborn\n");
} else if (age >= 65) {
```

```
    printf("You are a senior\n");
} else if (age >= 18) {
    printf("You are an adult\n");
} else {
    printf("You are a child\n");
}
```

Critical Rule: Order matters! Put the most specific conditions FIRST. C stops checking as soon as it finds a true condition. If age >= 18 came before age >= 65, a 70-year-old would print "adult" instead of "senior".

4. Comparison Operators

These are the operators you use inside if conditions:

Operator	Meaning	Example	Result
==	Equal to	`age == 18`	true if age is exactly 18
!=	Not equal to	`age != 18`	true if age is anything but 18
>	Greater than	`age > 18`	true if age is 19 or more
<	Less than	`age < 18`	true if age is 17 or less
>=	Greater or equal	`age >= 18`	true if age is 18 or more
<=	Less or equal	`age <= 18`	true if age is 18 or less

Common Mistake: == is for comparison. = is for assignment. Writing if (age = 18) sets age to 18 instead of checking it. Always use == inside conditions.

5. Using Booleans in If Statements

When you have a bool variable, you can use it directly as the condition — no need to compare it to true or false.

```
#include <stdbool.h>

bool isStudent = true;

// Both of these work, but the first is cleaner:
if (isStudent) {                    // preferred
    printf("Student discount!\n");
}

// Checking for false:
if (!isStudent) {
    printf("No discount\n");        // ! means NOT
}
```

Tip: Use ! (exclamation) to mean NOT. So !isStudent means "if isStudent is false".

6. Using Strings in If Statements

You cannot compare strings with == in C. Instead, use strlen() to check the length, or strcmp() to compare two strings. Always include string.h.

```
#include <stdio.h>
#include <string.h>

char name[30] = "";

printf("Enter your name: ");
fgets(name, sizeof(name), stdin);
name[strlen(name) - 1] = '\0';   // remove trailing newline

if (strlen(name) == 0) {
    printf("You did not enter your name\n");
} else {
    printf("Hello, %s!\n", name);
}
```

> **Remember:** Never use == to compare strings in C. Use strlen() to check if empty, or strcmp() to compare two strings (covered in a later lesson).

7. Practice Problem

> **Practice: Grade Checker**

Write a C program that asks the user to enter a score (0-100) and prints the corresponding grade and a message.

Requirements:

- 90 and above → Grade A — "Excellent!"
- 80 to 89 → Grade B — "Good job!"
- 70 to 79 → Grade C — "You passed."
- 60 to 69 → Grade D — "Needs improvement."
- Below 60 → Grade F — "You failed."
- Below 0 or above 100 → "Invalid score."

Expected Output (for input: 85)

```
Enter your score: 85
Grade: B
Good job!
```

> **Hint:** Check for invalid scores first (< 0 or > 100), then check from highest to lowest grade. Order matters!